

```
$ git --version 2.5+
```

Git Worktree

ทำงานหลาย branch พร้อมกัน โดยไม่ต้องสลับไปมา

```
$ git worktree add ../hotfix -b hotfix/login main
```

ต้นทุนของการสลับ branch

กำลังทำ feature อยู่ครึ่งทาง — มี hotfix ด่วนเข้ามา ต้องสลับไปแก้บน branch อื่น

งานที่ยังไม่เสร็จต้องถูก commit ทิ้งไว้ก่อน (WIP commit) แล้วค่อยแกะกลับมาทำต่อภายหลัง

dependencies และ build cache ถูกเขียนทับทุกครั้งที่ checkout ข้าม branch

```
$ git branch --show-current
feature/checkout-v2
$ git commit -am "WIP: ยังไม่เสร็จ"
$ git checkout hotfix/login
$ npm install && npm run build
# ...รอ build 10 นาที
$ git checkout feature/checkout-v2
$ git reset HEAD~1
# ...แล้ว context เมื่อที่อยู่ไหนนะ
```

Git Worktree คืออะไร

1 repository → หลาย working directory ที่ checkout branch ต่างกัน และทำงานพร้อมกันได้

SHARED

ทุก worktree ใช้ object database และ refs ชุดเดียวกัน — commit จากที่หนึ่ง เห็นได้ทันทีจากอีกที่

ISOLATED

แต่ละ directory มีไฟล์, index และ HEAD ของตัวเอง — แก้ไขงานคนละ branch โดยไม่รบกวนกัน

CHEAP

ไม่ต้อง clone ซ้ำ — สร้างได้ในไม่กี่วินาที ใช้พื้นที่เพิ่มเฉพาะไฟล์ working copy

repo ตัวอย่างของเรา มี 3 branch

```
$ git branch
* main
  hotfix/login
  pr-123
# * คือ branch ที่ checkout อยู่ตอนนี้
```

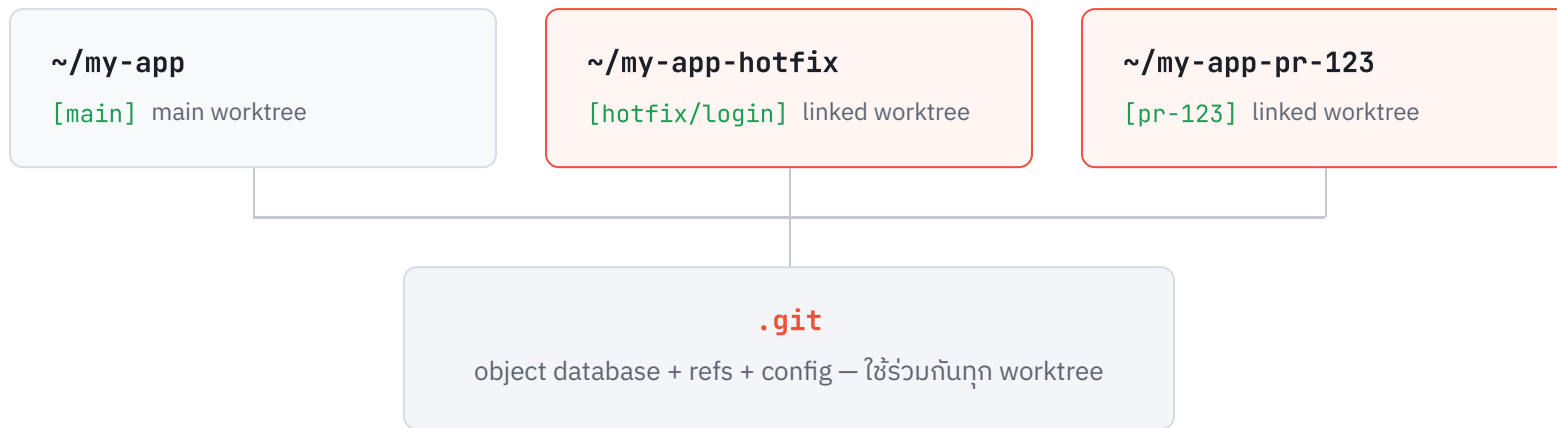
main branch หลักของโปรเจกต์ — โค้ดที่ deploy จริง

hotfix/login แก้บั๊กส่วนเรื่องระบบ login

pr-123 branch จาก pull request #123 ที่ดึงมา

สาม branch นี้จะเป็นตัวอย่างของเราตลอดทั้งเด็ค — สไลด์ถัดไปจะ
ทางทั้งสามออกเป็น worktree คนละ folder

โครงสร้างของ worktree



```
# main อยู่ที่ ~/my-app (repo เดิม) อยู่แล้ว — ไม่ต้องสร้างเพิ่ม
# สร้าง folder ให้อีกสอง branch ที่เหลือ
$ git worktree add ../my-app-hotfix hotfix/login
$ git worktree add ../my-app-pr-123 pr-123
```

กติกาสำคัญ: หนึ่ง branch ถูก checkout ได้ในหนึ่ง worktree เท่านั้น — ป้องกันการแก้ไข branch เดียวกันซ้อนกันสองที่

การสร้าง worktree — git worktree add

จากสไลด์ที่แล้ว — folder ~/my-app-hotfix และ ~/my-app-pr-123 ถูกสร้างด้วยคำสั่งนี้เอง มาดูรูปแบบการใช้งานทั้ง 3 แบบ

```
$ git worktree add ../hotfix hotfix/login  
# checkout branch ที่มีอยู่แล้ว ไปยัง directory ใหม่
```

```
$ git worktree add -b fix/crash ../fix-crash main  
# สร้าง branch ใหม่ชื่อ fix/crash จาก main พร้อม directory
```

```
$ git worktree add --detach ../experiment HEAD~3  
# detached HEAD — เหมาะกับการทดสอบ commit แก่แบบชั่วคราว
```

รูปแบบคำสั่ง: `git worktree add <path> <branch>` — path คือที่อยู่ directory, branch คือสิ่งที่จะ checkout

การจัดการ worktree — list / remove / prune

```
$ git worktree list
/Users/dev/my-app          a1b2c3d [main]
/Users/dev/my-app-hotfix   e4f5a6b [hotfix/login]
/Users/dev/my-app-pr-123   c7d8e9f [pr-123]
```

```
$ git worktree remove ../my-app-hotfix
# ลบ worktree ที่เลิกใช้ — มีไฟล์ค้างอยู่ต้องใช้ --force
```

```
$ git worktree prune
# เก็บกวาด metadata ของ worktree ที่ directory ถูกลบไปแล้ว
```

Workflow ตัวอย่าง — hotfix ด่วน

1 สร้าง worktree จาก main

```
$ git worktree add ../hotfix -b hotfix/login  
main
```

2 แก้ไข ทดสอบ และ push จากใน worktree

```
$ cd ../hotfix  
$ git commit -am "fix: login redirect"  
$ git push -u origin hotfix/login
```

3 กลับไปทำ feature ต่อได้ทันที

directory เดิมไม่ถูกแตะต้อง — ไฟล์ที่แก้ค้างไว้, dependencies และ build cache อยู่ครบทุกอย่าง

4 เสร็จแล้วเก็บกวาด

```
$ git worktree remove ../hotfix  
$ git branch -d hotfix/login
```

Use case — รัน AI coding agent หลายตัวพร้อมกัน

แต่ละ agent ได้ working directory และ branch ของตัวเอง — ทำงานขนานกันโดยไม่แย่งเขียนไฟล์ แล้วค่อย review และ merge ทีละ branch

```
~/agent-auth
```

```
[agent/auth]
```

```
$ claude "แก้ระบบ OAuth"
```

```
# agent 1 กำลังทำงาน...
```

```
~/agent-search
```

```
[agent/search]
```

```
$ claude "เพิ่ม fuzzy search"
```

```
# agent 2 กำลังทำงาน...
```

```
~/agent-tests
```

```
[agent/tests]
```

```
$ claude "เขียน test ให้ครบ"
```

```
# agent 3 กำลังทำงาน...
```

```
$ git worktree add ../agent-auth -b agent/auth main
```

```
# ทำซ้ำต่อ agent หนึ่งตัว — หนึ่ง worktree หนึ่ง branch
```

Worktree เทียบกับทางเลือกอื่น

	สลับ branch ใน directory เดียว	clone หลายชุด	git worktree
ทำงานหลาย branch พร้อมกัน	ไม่ได้	ได้	ได้
เวลา setup	ทันที	ช้า — ดาวน์โหลดทั้ง repo	ไม่กี่วินาที
disk space	น้อยที่สุด	เต็ม repo ต่อชุด	เฉพาะ working copy
branch / refs ตรงกันเสมอ	ใช่ — repo เดียว	ต้อง fetch / push เอง	ใช่ — แชร์ refs กัน
เหมาะกับ	งานเดียวตามลำดับ	โปรเจกต์แยกขาดถาวร	งานขนานใน repo เดิม

ข้อควรระวังและข้อจำกัด

หนึ่ง branch ต่อหนึ่ง worktree

checkout branch ที่ถูกใช้อยู่ใน worktree อื่นจะได้ error:
'branch is already checked out'

untracked files ไม่ตามไปด้วย

node_modules, .env และไฟล์ config ต้อง setup ใหม่ในแต่ละ worktree — 費เวลา install ไปด้วย

อย่าลบ directory ตรง ๆ

ลบด้วย rm จะเหลือ metadata ค้างใน .git — ใช้ git worktree remove หรือตามด้วย prune เสมอ

submodules รองรับได้จำกัด

repo ที่พึ่งพา submodule นึก ๆ ควรทดสอบ workflow ก่อนใช้จริง

สรุปและ cheat sheet

ครั้งหน้าที่มีงานด่วนแรก — สร้าง worktree แทนการสลับ branch แล้วงานเดิมจะอยู่ตรงที่เราทิ้งไว้เสมอ

```
$ git worktree add <path> <branch> # checkout branch เดิม
$ git worktree add -b <new> <path> <base> # สร้าง branch ใหม่
$ git worktree list # ดู worktree ทั้งหมด
$ git worktree remove <path> # ลบอย่างถูกวิธี
$ git worktree prune # เก็บกวาด metadata
```